

PPT-LAT-L1-ABI — v0 draft

The Layer-1 (math core) C ABI contract for Shannon-Prime. Locks the boundary between lib-shannonprime (C/CUDA/Vulkan/HVX) and any L2 binding (primary: Rust). Designed against the three tear-down axes: caller-allocates memory ownership, Send-but-not-Sync session handle, and an error surface that names VHT2 / Spinor / Frobenius / ARM / sieve failure modes explicitly. After this contract is signed off, .sp-model byte layout falls out mechanically.

1. Object lifetimes

Two opaque types cross the FFI:

```
typedef struct sp_model    sp_model;    // read-only after load; many sessions per model
typedef struct sp_session  sp_session;  // single-thread state: KV + ARM + sieve + arch s
```

Construction is L1's responsibility (only L1 knows the internal layout). Destruction is the caller's responsibility but always via the matched destroyer; the caller never free()s a returned pointer directly.

```
sp_status sp_model_load      (const char* sp_model_path,
                             const char* sp_tokenizer_path,
                             /*out*/ sp_model** out);
void      sp_model_unload    (sp_model*);

sp_status sp_session_create (const sp_model*,
                             const sp_session_config*,
                             volatile _Atomic bool* cancel_flag, // L2-owned, see §5
                             /*out*/ sp_session** out_session);
void      sp_session_destroy(sp_session*);
```

Cancel-flag lifetime contract. The cancel_flag pointer must remain valid (and the memory backing it must not be freed) for the entire lifetime of the sp_session — i.e. until sp_session_destroy returns. L2 owns the atomic; L1 only reads it. There is no cancel handle that L1 allocates, so there is no UAF window between a session being destroyed on one thread and a stale cancel handle being used on another. The L2 wrapper holds the atomic inside an Arc (see §5 and §8) which guarantees the address stays live for as long as either the session or any cancel-clone is alive.

2. Memory ownership — caller-allocates everywhere on the hot path

Every buffer crossing the FFI on the per-step path is caller-allocated. L1 never mallocs anything L2 has to free. Sizing comes from sp_model_arch before session creation:

```
sp_status sp_model_arch (const sp_model*, /*out*/ sp_arch_info*);
```

sp_arch_info carries vocab_size, hidden_dim, n_layers, n_heads, n_kv_heads, head_dim, rope_base_microcents, swa_window, ffn_variant, norm_variant, tied_embeddings, and the arch enum. L2 reads it once at load, sizes the logits buffer as vocab_size * sizeof(float), and never re-queries.

The out parameter is caller-stack-allocated by C convention; L1 populates the struct in place. Rust bindings should declare this as &mut MaybeUninit<sp_arch_info> rather than &mut sp_arch_info so the borrow checker doesn't assume the struct is initialized on

entry to the call (it isn't — L1 fills it). After `sp_status == SP_OK`, the binding may call `MaybeUninit::assume_init`.

The only L1-allocated memory crossing the boundary is the three opaque handles above, each with a matched destroyer. There is no `sp_alloc / sp_free` pair in the public ABI — internal arenas (activation, ARM bank, sieve, Spinor pool) are session-private and released by `sp_session_destroy`.

3. Forward pass — two-function ABI

```
sp_status sp_prefill_chunk (sp_session*,
                           const int32_t* tokens, size_t n_tokens,
                           /*out, caller-allocated*/ float* logits_last,
                           size_t logits_capacity);

sp_status sp_decode_step   (sp_session*,
                           int32_t token,
                           /*out, caller-allocated*/ float* logits,
                           size_t logits_capacity);
```

`sp_prefill_chunk` consumes `n_tokens` tokens, advances internal position by `n_tokens`, writes the last token's logits only. `sp_decode_step` consumes one token, advances by one, writes that token's logits. Two functions because their cost shape is asymmetric (compute-bound vs bandwidth-bound) and L2 must be free to interleave them across requests. `logits_capacity < vocab_size` returns `SP_EBADARG`.

4. Session manipulation — speculative-decoding-shaped from day one

```
sp_status sp_session_clone (const sp_session*,
                           volatile _Atomic bool* cancel_flag,    // L2-owned, see §5
                           /*out*/ sp_session** out);
sp_status sp_session_rewind (sp_session*, size_t n_tokens);
sp_status sp_session_position (const sp_session*, /*out*/ size_t* pos);
```

`sp_session_clone` is the spec-decode fork primitive: deep-copy KV + ARM + sieve, return an independent session. `sp_session_rewind` is the spec-decode reject primitive: roll back `n_tokens` of accepted state. ARM writes are journaled per-token so rewind is precise, not “drop the whole bank.” Getting these in v0 is what makes spec-decode tractable.

5. Cancellation — L2-owned atomic flag, no L1 call

There is no `sp_session_cancel()` function. The cancel surface is the volatile `_Atomic bool* cancel_flag` pointer L2 passes into `sp_session_create` (§1). L2 owns the storage; L1 only reads it.

```
// L2 side — sketch
let flag: Arc<AtomicBool> = Arc::new(AtomicBool::new(false));
let flag_ptr: *const AtomicBool = &*flag;           // stable for the Arc's lifetime
let sess = sp_session_create(model, &cfg, flag_ptr.cast(), &mut out_sess);

// HTTP-handler thread holds a clone of the Arc and cancels by:
flag_clone.store(true, Ordering::SeqCst);           // pure Rust, no FFI call
```

L1 reads the flag at every layer boundary in `sp_prefill_chunk` and every step in `sp_decode_step`. On observing true, the in-flight L1 call unwinds to the last completed boundary and returns `SP_ECANCEL`. Session state at that point is consistent (last layer fully applied), not partial.

Why this shape is UAF-proof. A naive design with an L1-allocated `sp_cancel*` paired to a session creates a window:

1. Worker thread drops the Session, `sp_session_destroy` runs, frees the C-side cancel handle.
2. Handler thread, milliseconds later, calls `sp_session_cancel(stale_ptr)` — dangling pointer dereference.

Inverting ownership eliminates the window. The atomic lives inside an `Arc<AtomicBool>`; the heap allocation backing an `Arc` never moves and is only freed when the *last* clone drops. Both the Session wrapper and the Cancel wrapper hold a clone, so the address L1 reads from remains valid until both are gone. Setting the flag on a already-destroyed-session `Arc` is a harmless write to memory nobody is reading anymore.

The cancel-flag read pattern in L1 is `__atomic_load_n(flag, __ATOMIC_RELAXED)` at boundaries (or `_InterlockedOr` on MSVC) — no fence required; the boundary itself is the synchronization point.

6. Determinism

Set at session create, immutable for the session’s lifetime:

```
typedef struct {
    size_t    max_context;
    bool      deterministic;           // serial reductions, single stream, no atomic-add
    uint32_t  arm_bank_kb;            // 0 = arch default
    uint32_t  sieve_capacity;         // 0 = arch default
    uint32_t  flags;
} sp_session_config;
```

`deterministic=true` is what `T_FRO_4` runs against (bit-exact gate). Production runs with `deterministic=false` (ULP-tolerance gate). Toggling mid-session is forbidden because reduction order and stream topology are baked into kernel selection at create time.

7. Error surface

`sp_status` is a signed int. `SP_OK` = 0. Negative = error. Positive reserved for future “soft” signals (e.g. “ARM bank approaching capacity, consider a write_stride bump”). All failing calls also set a thread-local error string retrievable via:

```
const char* sp_last_error(void);    // pointer valid until next L1 call on this thread
```

Enum (covers the VHT2 / Spinor decompression surface explicitly):

```
typedef enum {
    SP_OK                      = 0,

    // Generic
    SP_ENOMEM                  = -1,
    SP_ECANCEL                  = -2,
    SP_EBADARG                  = -3,
    SP_EBADSTATE                = -4,
```

```

SP_EUNSUPPORTED      = -5,
SP_EIO                = -6,

// Model load / arch
SP_EBADFORMAT        = -10,    // sp-model magic/version mismatch
SP_EBADARCH          = -11,    // arch_id not recognized
SP_ETOKENIZER_HASH   = -12,    // sp-tokenizer sha256 ≠ sp-model.tokenizer_hash
SP_EVOCAB             = -13,    // tokenizer vocab size ≠ model vocab size

// Discrete algebra layer – the "we lost the algebraic invariant" surface
SP_ESPINOR_BADBLOCK  = -20,    // 63-byte Spinor block parity/CRC mismatch
SP_EVHT2_DOMAIN      = -21,    // VHT2 inverse out-of-range
SP_EMOBIUS_PERM      = -22,    // Möbius reorder index invalid
SP_EOK_NORM          = -23,    // O_K element norm overflow
SP_EFROBENIUS_QUANT  = -24,    // Frobenius dequant scale/shift invalid
SP_ENTT_OVERFLOW     = -25,    // CRT NTT residue overflow (defensive – should be impossible)
SP_ERING_DEGREE      = -26,    // R_q polynomial degree mismatch

// Lattice / framework features
SP_ESIEVE_FULL       = -30,    // Friedman sieve full + eviction policy refused
SP_EARM_BANK_FULL    = -31,    // ARM HRR bank exhausted
SP_EDOMINANCE_CYCLE  = -32,    // □_d encountered a non-wqo input (corrupt KSTE)
SP_ECONTEXT_FULL     = -33,    // sequence position == max_context; L2 should trigger
                                // Fibonacci sub-sampling eviction and retry

// Backend
SP_ECUDA              = -40,    // wraps any cudaError_t; sp_last_error has the detail
SP_EVULKAN            = -41,
SP_EHVX              = -42,
SP_EBACKEND_OOM       = -43,    // device-side OOM, distinct from host SP_ENOMEM
} sp_status;

```

The discrete-algebra block (−20..−26) is the one that matters for correctness verification. Every gate in PPT-LAT-Theory T1..T7 / E9.x / E10 maps to one of those return codes if it trips at runtime.

SP_ECONTEXT_FULL is structurally distinct from SP_ENOMEM: the former is “the sequence position counter hit max_context”, and L2’s correct response is to trigger Fibonacci sub-sampling eviction (per Roadmap §20.x golden-ratio KV retention) on the session’s KV + ARM arenas and reissue the prefill/decode call. The latter is “the host allocator returned NULL” and is a hard fatal. Collapsing them would make the eviction-on-context-full policy unimplementable from L2.

8. Threading model — what L2’s Rust wrapper looks like

```

use std::sync::Arc;
use std::sync::atomic::{AtomicBool, Ordering};

pub struct Model { ptr: *const sp_model }

pub struct Session {
    ptr: *mut sp_session,
    cancel: Arc<AtomicBool>,    // held so the storage L1 reads stays live
}

```

```

}

#[derive(Clone)]
pub struct Cancel(Arc<AtomicBool>); // a clone of Session.cancel

impl Cancel {
    pub fn cancel(&self) { self.0.store(true, Ordering::SeqCst); }
}

impl Session {
    pub fn cancel_handle(&self) -> Cancel { Cancel(self.cancel.clone()) }
}

unsafe impl Send for Model {} unsafe impl Sync for Model {}
unsafe impl Send for Session {} // not Sync
// Cancel is Send + Sync automatically (Arc<AtomicBool> is Send + Sync)

```

Model is Send + Sync because it is immutable after `sp_model_load` returns. Session is Send (workers can hand it between threads between calls) but **not** Sync (no two threads may be inside `sp_prefill_chunk` / `sp_decode_step` on the same session simultaneously — enforced by `&mut self` on every step method). Cancel derives Send + Sync from its `Arc<AtomicBool>` field — no unsafe impls required, and no FFI call inside `Cancel::cancel`.

The HTTP handler thread holds a `Cancel`. The inference worker thread holds the `Session`. Both internally point at the same `Arc<AtomicBool>` storage, which L1 reads via the raw pointer it was given at `sp_session_create`. The Arc’s allocation is stable for as long as either side holds a clone, so the L1-side pointer never dangles. Cancellation from L3 is a single relaxed atomic store; no lock, no context switch, no FFI crossing.

9. What this contract does *not* commit to

Out of scope for v0, deliberate:

- **Streaming logits** — L2 may want token-by-token streaming callbacks. For v0 the caller polls. A callback variant (`sp_decode_step_cb`) can be added without breaking this ABI.
- **Multi-session batching at L1** — v0 is one-session-per-call. Cross-session prefill batching (vLLM-style continuous batching) is a v1 concern and goes in a new function, not a modification of `sp_prefill_chunk`.
- **In-process sampler** — L1 emits logits and stops. Any sampler (temperature, top-k, top-p, mirostat, grammar-constrained) is L2.
- **Tokenization** — L2 owns the SentencePiece / BPE blob via the `.sp-tokenizer` sibling file. L1 only verifies the hash.
- **Telemetry** — no `sp_metrics()` in v0. L2 instruments around the FFI boundary.

10. Sign-off checklist

Before this is locked, three reads against the tear-down axes:

1. **Memory ownership.** Grep this doc for any out parameter that isn’t either an opaque-handle constructor (`model/session/cancel`) or a caller-allocated buffer. There should be zero. If you find one, name it.
2. **Send/Sync compatibility.** Walk the four-line Rust block in §8. Every L1 function called from each thread must be safe given the marker traits asserted. Cancellation is the one that needs the hardest stare.

3. **Error surface vs failure modes.** Walk the eight load-bearing theorems (T1..T7 + E10) and confirm every runtime failure path from a theorem invariant has a named status code. Anything that would currently return SP_EBADSTATE and lose information is a gap.

Once those three are clean, .sp-model byte layout becomes:

```
magic          "SPMD"                // 4 bytes
version        u32                    // major.minor packed
arch_id        u32                    // sp_arch_info.arch_id
arch_struct    u8[256]                // memcpy'd into sp_arch_info
tokenizer_hash u8[32]                 // sha256 of paired .sp-tokenizer
tensor_count   u32
tensor_table   sp_tensor_entry[N]     // (offset, dtype_id, shape, name_hash)
tensor_data    [64-byte aligned; Spinor blocks pad 63→64]
```

— and is ~30 minutes of work.

Status. v0 draft. Not yet integrated into PPT-LAT-Systems §1 or the per-backend Phase-2 tracks. Sign-off goes against §10's three checks; on green, this becomes Appendix A of PPT-LAT-Systems and the L1 ABI is frozen for the duration of Phase 2.